PHYSICS 2VG3 Homework 3 2025/2026

Due: End of Day, Tuesday,24th February 2026, via Avenue

Note that this due date is after reading week but you are expected to work on this during the week on Feb 9-12.

Colliding Worlds

Complete your collision.py code as discussed in class…. (see Collisions.pdf Lecture notes). You should have begun with collision0.py from Avenue. In that file, the Particle class is set up as follows:

```
class Particle(NodePath):
    def __init__(self, *args, **kwargs):
        NodePath.__init__(self, *args, **kwargs)
        self.vel = Vec3(0.,0.,0.)
        self.inverseMass = 1.0
        self.radius = 1.0
```

The code is already set up to generate an off-centre collision (a bit different to the one below). You will need to add code to handle simple spherical collisions as discussed in lecture. When it works, test the code as follows:

Exercise 1: Off-centre collision between two masses

Create a system of two equal mass (1.0) particles. Place the first particle at x,y,z = (-5,25,0.4) with velocity v=(2,0,0) and the second at x,y,z = (5,25,-0.4) with velocity v=(-1,0,0). Run the test. Save the code as collision_2masses.py

Describe the outcome including the qualitative behaviour of the left and right masses after collision.

Extend your code to include a method calculateConservation that prints linear momentum, angular momentum and kinetic energy when called. Add code to make sure it is called at least once before and once after the collision. **Submit the extended code as collision_2mass.py**

Compute the initial and final momentum, energy and angular momentum before and after this collision. Report the numerical results in a pdf file to as many figures as printed by python print.

Exercise 2: A basketball, a ping pong ball and a wall

Create a system consisting of 3 spheres with masses 1.0, 100.0 and infinity (the wall). Set these using the inverse mass.

What is the inverse mass of the 3rd object and why does this make sense here?

Assign them radii of 1.0, 10.0 and 100.0 respectively. Put all the masses on a line at y=200.0, z=0.0, with x positions at -11.0, 0.0 and 120.0 respectively. Have the first two moving right at v_x=10.0 and no other motions. Run this test. **Submit this code as collision_3masses.py**

Describe what happens. What are the final velocities of all three masses?

Describe the sequence of events that is occurring to achieve this result.

Modify the set-up in minor ways (e.g. initial x positions, initial v_x for the two masses). Always keep the 3 masses in row in the same order with no initial overlap and always with the left two masses moving at the same speed. How does this affect the outcome?

What would be the limiting result if the first mass was made very light (without changing the radius)? You can experiment with your code to explore this. Express your answers for the three final velocities in terms of multiples of the initial v_x. That is – give an answer that would apply for any choice of $v_x > 0$.

Exercise 3: Generate code to collide 10 or more randomly placed particles within a 3D cubic region. Use the random module's uniform function in python to set things up, e.g.

```
import random

p.setPos(Vec3(random.uniform(-18,18),100,random.uniform(-18,18)))
p.vel = Vec3(random.uniform(-10,10),self.random.uniform(-10,10),random.uniform(-10,10))
```

Use the wall bounce technique from last week in all 3 directions (6 tests total) (see: Collisions pdf lecture notes). Put the walls at locations similar to +/- 20 in all directions from the centre of the view at (0,100,0). Submit this code as collsion_many.py.

Print out the conserved quantities at the start and after 30 seconds and report those in your hand-in pdf.

What is conserved now? If something is not conserved, explain why not.

Please combine all the files into a single zip and upload that as well if possible. So that means uploading them twice effectively.